

Как работают компоненты и пропсы в Астро

Оглавление

- Как работают компоненты и пропсы в Астро
 - Оглавление
- Проблема
- Цель
- Решение
 - Пропсы компонента
 - Компоненты
 - Структура компонента
 - Скрипт компонента
 - Шаблон компонента
 - Компонентный дизайн

- Слоты
- Именованные слоты
- Резервный контент для слотов
- Перенос слотов
- HTML-компоненты
- Дополнительно (глубже в кроличью нору)
 - Fragments
 - set:directives
 - framework-components
- [Материалы для статьи](#)

Проблема

передача информации между
файлами в Astro

Цель

- Разобраться с передачей пропсов
- Лучшие способы передачи
- Разбор компонентов в пропсах

Решение

Пропсы компонента

Компонент **Astro** может определять и принимать входные параметры — пропсы. Эти пропсы затем становятся доступными шаблону компонента для рендеринга HTML. Пропсы доступны в глобальном объекте ***Astro.props*** в вашем скрипте.

Вот пример компонента, который получает пропсы **greeting** и **name**.

Обратите внимание, что получаемые входные параметры деструктурированы из глобального объекта ***Astro.props***.

```
---  
// Применение:  
<GreetingHeadline  
  greeting="Привет"  
  name="Партнёр" />  
const { greeting, name } =  
  Astro.props;  
---  
<h2>{greeting}, {name}!  
</h2>
```

Этот компонент при импорте и визуализации в других компонентах **Astro**, макетах или страницах может передавать эти параметры в качестве атрибутов:

```
---
import GreetingHeadline
from
'./GreetingHeadline.astro';
const name = "Astro"
---
<h1>Поздравительная
открытка</h1>
<GreetingHeadline
greeting="Привет" name=
{name} />
<p>Надеюсь, у вас чудесный
день!</p>
```

Вы также можете определить свои пропсы с помощью **TypeScript** с интерфейсом типа **Props**. Astro автоматически подхватит интерфейс **Props** в вашем блоке метаданных и выдаст предупреждения/ошибки типа. Этим входным параметрам также могут

быть присвоены значения по умолчанию при деструктуризации из ***Astro.props***.

```
---  
interface Props {  
  name: string;  
  greeting?: string;  
}  
  
const { greeting =  
  "Привет", name } =  
  Astro.props;  
---  
<h2>{greeting}, {name}!  
</h2>
```

Пропсам компонента могут быть присвоены значения по умолчанию, которые можно использовать, если они не указаны.

```
---  
const { greeting =  
  "Привет", name =  
  "Астронавт" } =  
  Astro.props;  
---  
<h2>{greeting}, {name}!  
</h2>
```

Компоненты

Структура компонента

Компонент **Astro** состоит из двух основных частей: Скрипта компонента и Шаблона компонента. Каждая часть выполняет свою работу, но вместе они обеспечивают структуру, которая одновременно проста в использовании и достаточно выразительна, чтобы справиться с тем, что вы захотите создать.

```
---  
// Скрипт компонента  
(JavaScript)  
---  
<!-- Шаблон компонента  
(HTML + выражения JS) -->
```

Скрипт компонента

Astro использует блок кода (**---**) для идентификации скрипта в вашем компоненте **Astro**. Если вы когда-либо писали Markdown раньше, возможно, вы уже знакомы с похожей концепцией, называемой метаданные. Идея **Astro** о скрипте компонента была непосредственно вдохновлена этой концепцией.

Вы можете использовать скрипт компонента для написания любого кода **JavaScript**, необходимого для визуализации шаблона. Это может включать в себя:

импорт других компонентов **Astro** импорт компонентов из других фреймворков, таких как **React** импорт данных, например файла **JSON** получение контента из **API** или базы данных создание переменных, на которые вы будете ссылаться в своем шаблоне

```
---  
import SomeAstroComponent  
from  
'../components/SomeAstroCom  
ponent.astro';  
import SomeReactComponent  
from
```

```
'../components/SomeReactComponent.jsx';
import someData from
'../data/pokemon.json';

// Доступ к пропсам
компонента, таким как `
```

Блок кода предназначен для того, чтобы гарантировать, что написанный в нём

JavaScript остаётся «изолированным». Он не попадёт в ваше фронтенд-приложение и не окажется в руках пользователя. Здесь можно безопасно писать код, который является ресурсоёмким или чувствительным (например, подключение к вашей приватной базе данных), не беспокоясь о том, что он когда-либо попадёт в браузер пользователя.

Шаблон компонента

Шаблон компонента следует за блоком кода и определяет вывод **HTML** вашего компонента.

Если вы напишете здесь простой **HTML**, ваш компонент будет отображать этот **HTML** на любой странице Astro, в которую он импортирован и используется.

Однако, синтаксис **Astro** так же поддерживает **JavaScript** выражения, теги **<style>** и **<script>** **Astro**, импортированные компоненты, и специальные директивы **Astro**. Данные и значения, определённые в скрипте компонента, можно использовать в шаблоне компонента для создания динамически создаваемого **HTML**.

```
---  
// Здесь ваш скрипт  
компонента!  
import Banner from  
'../components/Banner.astro'  
;  
import Avatar from  
'../components/Avatar.astro'  
;  
import  
ReactPokemonComponent from  
'../components/ReactPokemon
```

```
Component.jsx';  
const myFavoritePokemon =  
  [/* ... */];  
const { title } =  
  Astro.props;  
---  
<!-- Поддерживаются HTML  
комментарии! -->  
{/* Синтаксис комментариев  
JS также действителен! =>  
скорее jsx */}  
  
<Banner />  
<h1>Привет, мир!</h1>  
  
<!-- Используем пропсы и  
другие переменные из  
скрипта компонента: -->  
<p>{title}</p>  
  
<!-- Осуществляем  
отложенный рендеринг  
компонента с резервным  
контентом для загрузки: -->
```

```
<Avatar server:defer>
  <svg slot="fallback"
class="generic-avatar"
transition:name="avatar">..
.</svg>
</Avatar>
```

```
<!-- Включаем другие
компоненты
пользовательского
интерфейса с помощью
директивы `client`: для
гидратации: -->
<ReactPokemonComponent
client:visible />
```

```
<!-- Комбинируем HTML с
выражениями JavaScript, как
в JSX: -->
<ul>
```

```
{myFavoritePokemon.map((dat
a) => <li>{data.name}
</li>))}
```

```
</ul>
```

```
<!-- Используем директиву  
шаблона для создания имён  
классов из нескольких строк  
или даже объектов! -->
```

```
<p class:list={["add",  
"dynamic", {classNames:  
true}]} />
```

Компонентный дизайн

Компоненты спроектированы так, чтобы их можно было повторно использовать и компоновать. Вы можете использовать компоненты внутри других компонентов для создания более продвинутого пользовательского интерфейса.

Например, компонент **Button** можно использовать для создания компонента **ButtonGroup**:

```
---  
import Button from  
'./Button.astro';  
---  
<div>  
  <Button title="Кнопка 1"  
>  
  <Button title="Кнопка 2"  
>  
  <Button title="Кнопка 3"  
>  
</div>
```

Слоты

`<slot />` это заполнитель для внешнего HTML-содержимого, позволяющий вставлять дочерние элементы из других файлов в шаблон компонента.

По умолчанию все дочерние элементы, переданные компоненту, будут отображаться в его **<slot />**.

Заметка

В отличие от пропсов, которые являются атрибутами, передаваемыми компоненту **Astro**, доступными для использования во всем вашем компоненте с помощью **Astro.props**, слоты отображают дочерние элементы **HTML** там, где они написаны.

- `src/components/Wrapper.astro`

```
---  
import Header from  
'./Header.astro';  
import Logo from  
'./Logo.astro';  
import Footer from  
'./Footer.astro';
```

```

const { title } =
  Astro.props
---
<div id="content-wrapper">
  <Header />
  <Logo />
  <h1>{title}</h1>
  <slot />  <!-- дочерний
элемент будет здесь -->
  <Footer />
</div>

```

- src/pages/fred.astro

```

---
import Wrapper from
  '../components/Wrapper.astro';
---
<Wrapper title="Fred's
Page">

```

```
<h2>Всё о Фреде</h2>  
<p>Вот немного информации  
о Фреде.</p>  
</Wrapper>
```

Этот шаблон является основой компонента макета **Astro**: целая страница **HTML**-контента может быть «обёрнута» тегами и передана компоненту для отрисовки внутри определённых в нем общих элементов страницы.

Именованные слоты

Компонент **Astro** также может иметь именованные слоты. Это позволяет передавать в местоположение слота только элементы **HTML** с соответствующим именем слота.

Слоты именуются с помощью атрибута ***name***:

- src/components/Wrapper.astro

```
---
import Header from
'./Header.astro';
import Logo from
'./Logo.astro';
import Footer from
'./Footer.astro';

const { title } =
  Astro.props
---
<div id="content-wrapper">
  <Header />
  <!-- дочерние элементы с
атрибутом `slot="after-
header"` будут находиться
здесь -->
  <slot name="after-
```

```
header"/>
  <Logo />
  <h1>{title}</h1>
  <!-- дочерние элементы
без `slot`, или с
`slot="default"` будут
находиться здесь -->
  <slot />
  <Footer />
  <!-- дочерние элементы с
атрибутом `slot="after-
footer"` будут находиться
здесь -->
  <slot name="after-
footer"/>
</div>
```

Чтобы внедрить **HTML**-контент в определённый слот, используйте атрибут **slot** в любом дочернем элементе, чтобы указать имя слота. Все остальные дочерние элементы компонента будут

вставлены в стандартный (безымянный) `<slot />`.

- `src/pages/fred.astro`

```
---
import Wrapper from
'../components/Wrapper.astro';
---
<Wrapper title="Страница
Фреда">
  
  <h2>Всё о Фреде</h2>
  <p>Вот немного информации
о Фреде.</p>
  <p slot="after-
footer">Авторское право
2022</p>
</Wrapper>
```

-
- *** Совет Используйте атрибут ***slot="my-slot"*** в дочернем элементе, который вы хотите передать в соответствующий заполнитель ***<slot name="my-slot" />*** в вашем компоненте.

Чтобы передать несколько HTML-элементов в заполнитель ***<slot/>*** компонента без обёртывающего ***<div>***, используйте атрибут ***slot=""*** на компоненте ***<Fragment/>***

- `src/components/CustomTable.astro`

```
---  
//  Создание  
пользовательской таблицы с  
именованными заполнителями  
для заголовка и тела  
контента
```

```

---
<table class="bg-white">
  <thead class="sticky top-0 bg-white"><slot
name="header" /></thead>
  <tbody class="[&_tr:nth-child(odd)]:bg-gray-100">
<slot name="body" />
</tbody>
</table>

```

Вставьте несколько строк и столбцов **HTML**-контента с помощью атрибута ***slot=""***, чтобы указать содержимое ***"header"*** и ***"body"***. Отдельные элементы **HTML** также можно стилизовать:

- `src/components/StockTable.astro`

```

---
import CustomTable from

```



```

'./CustomTable.astro';
---
<CustomTable>
  <Fragment slot="header">
<!-- передаём заголовок
таблицы -->
    <tr><th>Название
продукта</th><th>Единицы на
складе</th></tr>
  </Fragment>

  <Fragment slot="body">
<!-- передаём тело таблицы
-->
    <tr><td>Шлёпанцы</td>
<td>64</td></tr>
    <tr><td>Ботинки</td>
<td>32</td></tr>
    <tr><td>Кроссовки</td>
<td class="text-red-
500">0</td></tr>
  </Fragment>
</CustomTable>

```

Обратите внимание, что именованные слоты должны быть непосредственными дочерними элементами компонента. Вы не можете передавать именованные слоты через вложенные элементы.

- *** Совет

Именованные слоты также можно передавать в [компоненты UI фреймворков!](#)

- *** Заметка

Невозможно динамически генерировать имя слота **Astro**, например, внутри функции **map**. Если эта функция необходима в компонентах UI-фреймворка, лучше всего генерировать эти динамические слоты внутри самого фреймворка.

Резервный контент для слотов

Слоты также могут отображать резервный контент. Когда в слот не передаются соответствующие дочерние элементы, элемент `<slot />` отобразит свои собственные дочерние элементы-заполнители.

- `src/components/Wrapper.astro`

```
---  
import Header from  
  './Header.astro';  
import Logo from  
  './Logo.astro';  
import Footer from  
  './Footer.astro';  
  
const { title } =  
  Astro.props  
---
```

```
<div id="content-wrapper">
  <Header />
  <Logo />
  <h1>{title}</h1>
  <slot>
    <p>Это резервное
содержимое, если в слот не
передан дочерний
элемент</p>
  </slot>
  <Footer />
</div>
```

Резервный контент будет отображаться только в том случае, если нет соответствующих элементов с атрибутом ***slot="name"***, передаваемых в именованный слот.

Astro передаст пустой слот, если элемент слота существует, но не содержит контента для передачи. Резервный

контент нельзя использовать в качестве значения по умолчанию, когда передаётся пустой слот. Резервный контент отображается только тогда, когда элемент слота не найден.

- как по мне почти бессмысленно лучше уж рухнет если что... разве что если отображать и обрабатывать с базы данных... и вывести надпись данных нет но тут астро как по мне проигрывает next.js

Перенос слотов

Слоты можно передавать другим компонентам. Например, при создании вложенных макетов:

- `src/layouts/BaseLayout.astro`



```
---  
---  
<html lang="en">  
  <head>  
    <meta charset="utf-8"  
  />  
    <link rel="icon"  
type="image/svg+xml"  
href="/favicon.svg" />  
    <meta name="viewport"  
content="width=device-  
width" />  
    <meta name="generator"  
content={Astro.generator}  
  />  
    <slot name="head"/>  
  </head>  
  <body>  
    <slot />  
  </body>  
</html>
```

- src/layouts/HomeLayout.astro

```
---  
import BaseLayout from  
"./BaseLayout.astro";  
---  
<BaseLayout>  
  <slot name="head"  
slot="head"/>  
  <slot />  
</BaseLayout>
```

- Результат

```
<html lang="en">  
  <head>  
    <meta charset="utf-8"  
/>  
    <link rel="icon"  
type="image/svg+xml"  
href="/favicon.svg" />
```

```
<meta name="viewport"
content="width=device-
width" />
<meta name="generator"
content="Astro vX.Y.Z" />

<!-- Слот head пуст -->

</head>
<body>
  <!-- Слот по умолчанию
пуст -->
</body>
</html>
```

- *** Моя Заметка Может быть очень полезно для SEO
- *** Заметка

Именованные слоты могут быть переданы другому компоненту,

используя атрибуты ***name*** и ***slot*** в теге ***<slot />***

Теперь содержимое стандартного (безымянного) слота и слота head, переданных в HomeLayout, будет перенесено в родительский компонент BaseLayout.

- src/pages/index.astro

```
---
import HomeLayout from
"../layouts/HomeLayout.astro";
---
<HomeLayout>
  <title
slot="head">Astro</title>
  <h1>Astro</h1>
</HomeLayout>
```

- Результат

```
<html lang="en">
  <head>
    <meta charset="utf-8"
  />
    <link rel="icon"
type="image/svg+xml"
href="/favicon.svg" />
    <meta name="viewport"
content="width=device-
width" />
    <meta name="generator"
content="Astro vX.Y.Z" />

    <!-- Title "Astro"
попадает сюда благодаря
перенаправлению слотов -->
    <title>Astro</title>

  </head>
  <body>
    <!-- Заголовок "Astro"
```

попадает сюда как основной контент -->

```
<h1>Astro</h1>
</body>
</html>
```

HTML-компоненты

Astro поддерживает импорт и использование *.html* файлов в качестве компонентов или размещение этих файлов в подкаталоге *src/pages/* в качестве страниц. Возможно, вам захочется использовать **HTML**-компоненты, если вы повторно используете код с существующего сайта, созданного без фреймворка, или если вы хотите убедиться, что ваш компонент не имеет динамических функций.

HTML-компоненты должны содержать только валидный **HTML**, поэтому они лишены ключевых возможностей компонентов **Astro**:

Они не поддерживают метаданные, импорт на стороне сервера или динамические выражения. Любые теги **<script>** остаются неупакованными, и обрабатываются, как если бы у них был атрибут **is:inline**. Они могут только ссылаться на ресурсы, которые находятся в папке **public/**.

- *** Заметка

В **HTML**-компоненте элемент **<slot />** будет работать так же, как и в компоненте **Astro**. Чтобы использовать **HTML Web Component Slot**, добавьте атрибут **is:inline** к вашему элементу **<slot>**.

Дополнительно (глубже в кроличью нору)

Fragments

Astro поддерживает `<> </>` нотацию и предоставляет встроенный компонент. Этот компонент может быть полезен для предотвращения использования элементов-обёрток при добавлении `set:*directives` для внедрения **HTML**-строки.

В следующем примере текст абзаца визуализируется с использованием компонента:

```
---
const htmlString = '<p>Raw
HTML content</p>';
---
```

```
<Fragment set:html=  
{htmlString} />
```

set:directives

! NF!!!!

framework-components

! NF!!!!

Материалы для статьи

[Астро документация](#)

[set:directives](#)

[Astro-syntax](#)

[XSS](#)

Framework-components